

Introduction to Data Tools and Storage



SoftUni Team
Technical Trainers



SoftUni



Software University

<https://about.softuni.bg>

Have a Question?



sli.do

#ETL

Table of Contents

1. Pandas
2. PostgreSQL
3. AWS S3
 - Buckets and Objects
 - Use Cases
 - Data Engineering Pipelines
4. Data Warehouse
5. Snowflake





Pandas

The Python Library for Data Analysis

What is Pandas?

- What is **Pandas**?
 - Pandas is an **open-source Python library** that lets you **load**, **clean**, and **analyze** structured data using simple, intuitive Python code



Why Pandas? Key Features

- Built on top of NumPy for high performance
- Key Features
 - The **DataFrame** is the core 2D structure for tabular data
 - Reads **CSV, Excel, JSON, SQL** and many other formats
 - Built-in **cleaning, filtering, and aggregation**
 - Integrates with **NumPy, Matplotlib, and Scikit-learn**
 - Industry-standard for **data science workflows**

- Pandas provides **two core data structures** for working with data in Python and they make data **easy to load**, transform, and analyze
 - Series - A **Series** is a **one-dimensional labeled array** that can hold any data type. Think of it as a single **column** with an associated **index for each value**
 - DataFrame - A **DataFrame** is a **two-dimensional labeled table** like a spreadsheet, with rows and columns of any data type

- Pandas reads **many file formats** — CSV, Excel, JSON, SQL, Parquet, and more
- Each reader returns a **DataFrame** ready to query, transform, and analyze your data with simple, clean Python code

```
df = pd.read_csv('sales_data.csv')
```

```
df = pd.read_excel('report.xlsx', sheet_name='Q1')
```

```
df = pd.read_sql('SELECT * FROM users', conn)
```


- **Inspecting**

- **Quick Look:** After loading data, use built-in methods to quickly understand the size, structure, and content of any DataFrame before analysis

```
df.describe(include='all').head()  
# summary stats for all columns
```

- **Filtering**

- **Selecting Data:** Pandas makes it easy to *select columns and filter rows* using simple, intuitive boolean conditions and column names

```
high_value = df[(df['amount'] > 1000) &  
(df['country'] == 'BG')]
```

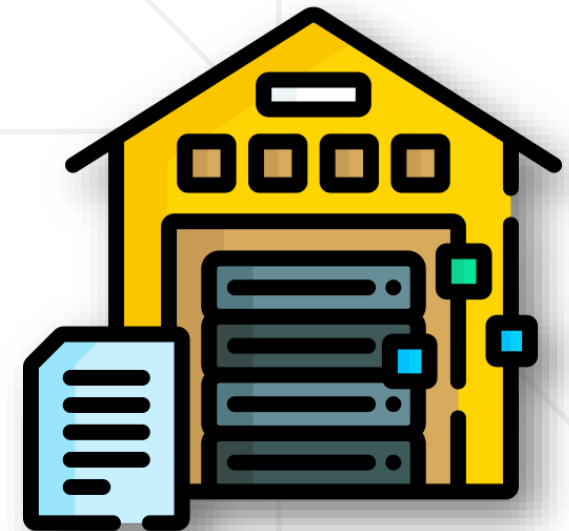
- **Grouping**

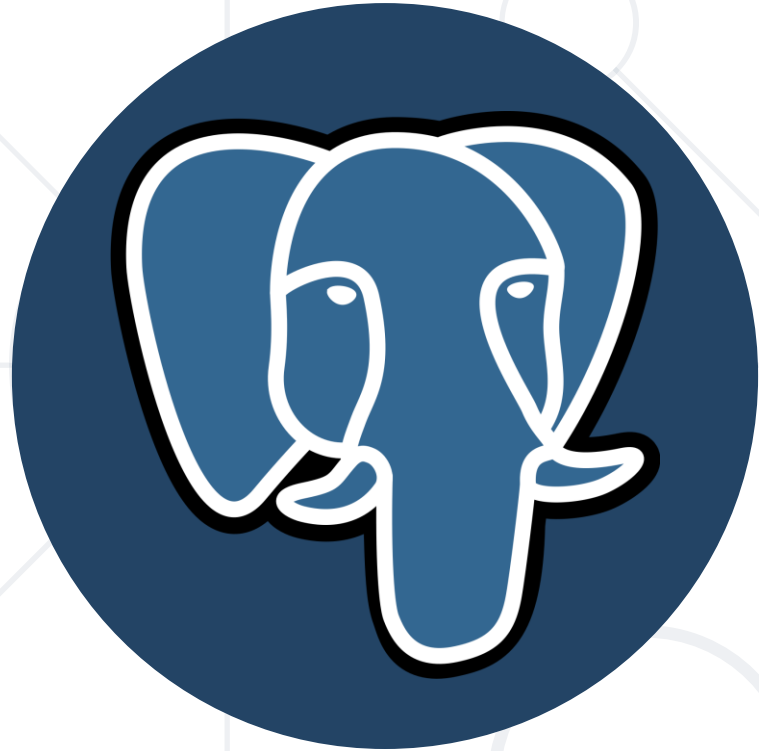
- **Aggregation:** Use `groupby()` to split data by category, then apply aggregations like `sum()`, `mean()`, or `count()` to summarize results

```
result =  
df.groupby('category')['sales'].sum().sort_values(ascending=False)
```

Combining and Reshaping DataFrames

- Combine DataFrames with `merge()` — like SQL JOIN
- Stack rows or columns with `concat()`
- Reshape data with `pivot()` and `melt()`





PostgreSQL

What is a Database?

- What is **Database**?
 - A Database is an **organized collection of data** managed by a **DBMS**, **stores**, and **queries** data efficiently, supports concurrent access, and enforces data integrity



What is PostgreSQL?

- What is **PostgreSQL**?
 - PostgreSQL is a **powerful, open-source object-relational database** (ACID-compliant, SQL-based) designed for **reliability, scalability, and extensibility**, supporting both **structured and JSON / semi-structured data**

- **Tables**

- Store data in rows and columns, like a spreadsheet
- Each row is a record; each column has a defined data type

- **Primary Keys**

- A unique identifier for each row in a table — guarantees no duplicates

- **Foreign Keys**

- References a primary key in another table — creates relationships between tables

- **Schemas**

- Logical groups of related tables, views, and other database objects

- SQL has four **core operations** — CRUD: Create, Read, Update, and Delete database records
- Common keywords combine **SELECT**, WHERE, **ORDER BY** and JOIN clauses for powerful queries

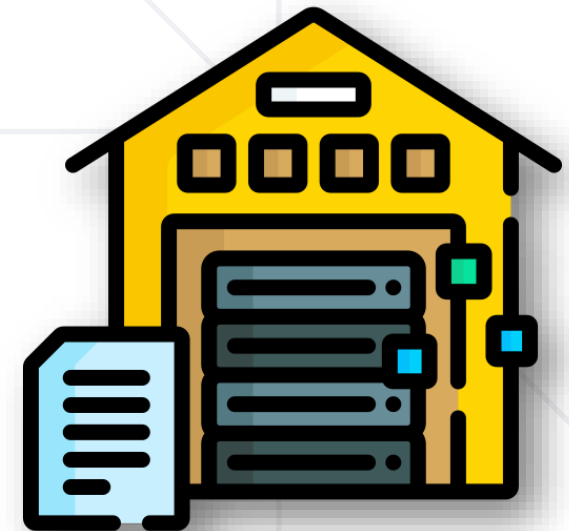
```
SELECT name, email FROM users WHERE country = 'BG';
```

```
INSERT INTO users (name, email) VALUES ('Ivan',  
                                         'ivan@x.bg');
```

```
UPDATE users SET active = false WHERE id = 42
```

- **JOINS**: Combine rows from two or more tables based on a related column
- **Most common types**: INNER, LEFT, RIGHT, and FULL OUTER JOIN
- Why JOINS matter
 - Normalize data — avoid duplicating information
 - Combine related data only when needed
 - Foundation of every relational query that spans multiple tables

- Indexes speed up SELECT queries dramatically
- Trade-off — slower INSERTs and extra storage
- Use EXPLAIN ANALYZE to see if a query uses an index
- **JSON** — like **JSONB** column types





AWS S3

Cloud Object Storage on Amazon Web Services

What is the Cloud?

- What is **the Cloud**?
 - The cloud is a **network of remote servers** that lets you **store**, **compute**, and **deliver** data and applications on demand, without owning physical hardware



- What is **AWS S3**?
 - Amazon S3 (Simple Storage Service) is a **highly scalable cloud object storage service** (part of Amazon Web Services) designed for **durability, availability, and virtually unlimited capacity**, able to store any **file type** and **from a few KB to terabytes**

- S3 storage is built on two key concepts - **Buckets and Objects** — together they form the foundation of any S3-based system.
Globally unique names and infinite scale
 - Buckets - A **bucket** is a **top-level container** for your files. It has a globally unique name and lives in a chosen **AWS region**. You can configure access permissions and **lifecycle rules**
 - Objects - An **object** is a **single file** — identified by a key (path-like name), with metadata, content, and a version ID

- **Common Use Cases**

- Backups and Disaster Recovery
- Data Lake — central staging area for all raw data
- Static Website Hosting and Asset Storage

- **In Practice**

- **Netflix** - Stores petabytes of video assets and serves them to millions of viewers worldwide
- **Airbnb** - Stores user-uploaded photos, listing data, and analytics logs
- **Spotify** - Uses S3 as the long-term archive for podcasts and music files

- **Data Lake Foundation**
 - S3 is the most common data lake storage layer
 - Stores raw data from many sources in its original format
- **ETL / ELT Pipelines**
 - Tools like Airflow, dbt, and Glue read from and write to S3
- **Integration with Snowflake**
 - Snowflake can directly query data stored in S3 via external tables
- **Cost Efficiency**
 - Pay only for what you store and transfer — cheaper than database storage



Data Warehouse

What is a Data Warehouse?

- What is a **Data Warehouse**?
 - A Data Warehouse is a **central system** that pulls data from **many sources, cleans and organizes it, and serves it for analysis** — it is built for fast queries on huge amounts of historical data



Why We Need More Than Databases

- **Operational databases** (like PostgreSQL) are great for running an application
- **But analytics needs more:** large-scale reporting, dashboards, and historical trend analysis
- Why we need something more
 - Operational databases struggle with billions of rows and complex aggregations
 - Mixing analytics with live transactions slows down the application
 - Enter the Data Warehouse — purpose-built for analytics at scale

- **Operational Database** — like a desk drawer for daily work
- **Data Warehouse** — like a research library archive
- Why two systems?
 - Different jobs need different tools — speed vs. depth
 - Mixing fast transactions with heavy analytics slows everything down
 - Specialized systems beat one-size-fits-all



Snowflake

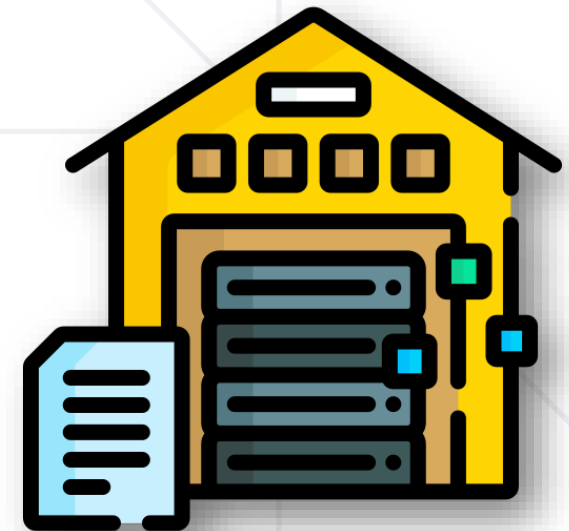
What is Snowflake?

- What is **Snowflake**?
 - Snowflake is a **modern cloud data warehouse** (runs on AWS, Azure, or Google Cloud) known for being **fully managed and easy to use, available 24/7 across continents, and ready for huge analytics workloads**, with no servers to set up — you just **log in and start querying your data**



Why Snowflake?

- Zero infrastructure to manage — fully managed in the cloud
- Pay only for what you use — auto-scales up and down on demand
- Works with all your data — structured, JSON, even files in S3



Key Terms You Will Hear

- **OLAP** — **analytics** on aggregated, historical data
- **ETL / ELT** — moving and transforming data between systems
- **BI** — **Business Intelligence** tools like Tableau, Power BI, Looker
- **Star Schema** — a common data warehouse table design pattern
- **Cloud Data Warehouse** — a Data Warehouse that runs as a fully managed cloud service

- Security and Governance
 - RBAC (Roles, Grants, Privileges)
 - **Role-Based Access Control (RBAC)** lets you assign permissions via **roles**
 - Dynamic Data Masking
 - Hides sensitive data from unauthorized users **in real time**
 - Row Access Policies
 - Filters **rows** returned based on user role or attributes
 - Column-Level Security
 - Restricts access to **specific columns** in a table
 - Data Classification (Sensitive Data Detection)
 - Helps identify **personally identifiable information (PII)** automatically

- **Virtual Warehouses:**
 - the compute resources (compute layer) in Snowflake
 - execute SQL queries, load data, and perform data transformations
 - completely separated from the storage layer, allowing compute resources and data storage to scale independently.
- **Key Features:**
 - Provide CPU and memory resources for query execution.
 - Can be started and stopped independently of the data.
 - Support multiple sizes (X-Small, Small, Medium, Large, and more).
 - Enable **Auto Suspend** and **Auto Resume** to optimize costs.
 - Multiple warehouses can operate simultaneously on the same data without resource contention.

- **Sources Layer**
 - Apps, websites, IoT, SaaS — where raw data is created
 - Typically stored in operational databases like PostgreSQL
- **Storage Layer**
 - Data Lake (AWS S3) for raw files; Data Warehouse (Snowflake) for query-ready data
- **Transform & Analyze Layer**
 - Pandas, dbt, SQL queries — turn raw data into clean, useful insights
- **Consume Layer**
 - Dashboards, BI tools, ML models — where humans and systems use the data

- Data Warehouse Layers in Snowflake
 - Staging Layer (Raw Ingest)
 - Raw Layer (Immutable Historical Store)
 - Cleansed Layer (Data Quality Checks)
 - Business / Dimensional Layer (Facts & Dimensions)
 - Presentation Layer (BI-ready)

- Master **Pandas** for **data manipulation** in **Python**
- Use **PostgreSQL** for relational data — tables, keys, joins, and indexes
- Understand **AWS S3** — **buckets, objects,** and **use cases** — building a **solid foundation** in **Data Warehouses** and **Snowflake**



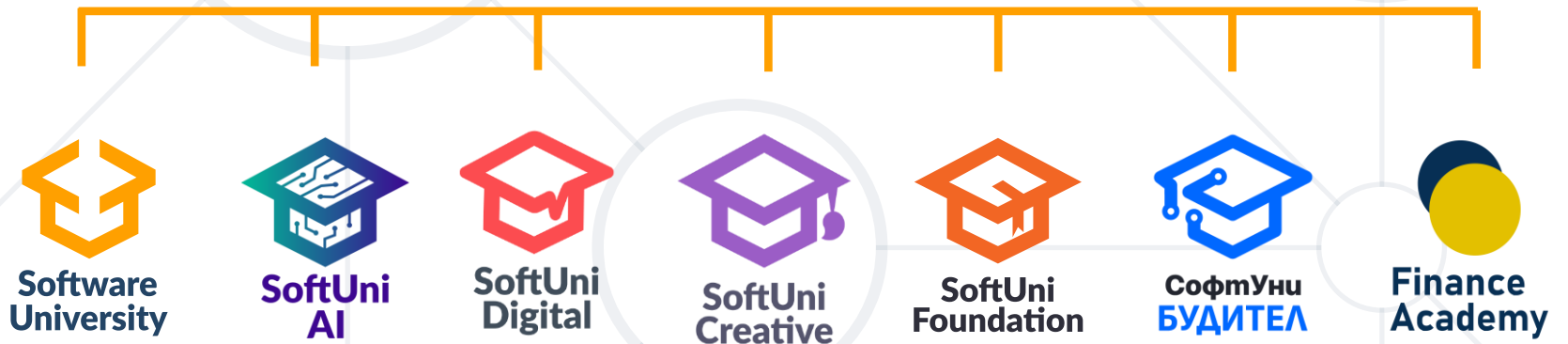
SoftUni Diamond Partners



Questions?



SoftUni



- Software University – High-Quality Education, Profession and Job for Software Developers

- softuni.bg, about.softuni.bg

- Software University Foundation

- softuni.foundation

- Software University @ Facebook

- facebook.com/SoftwareUniversity



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg/>
- © Software University – <https://softuni.bg>

